LESSON 05 · DOCKER · 50 MIN

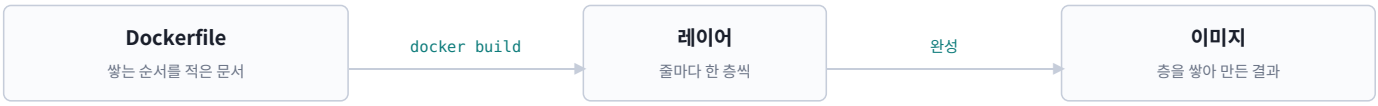
5교시 Dockerfile — 내 앱을 이미지로 만들기

Dockerfile은 위에서 아래로 층을 쌓습니다. 무엇을 위에 두느냐에 따라 빌드가 1초에 끝나기도, 30초가 걸리기도 합니다. 같은 수정을 두 가지 순서에 적용해 시간을 직접 재 봅니다.

개념 12분 · 실습 28분 · 정리와 질문 10분

OVERVIEW

Dockerfile 한 줄 = 레이어 한 층



> 이미 만든 층은 캐시에 남아 다음 빌드에서 재사용됩니다. 다만 **한 층이 바뀌면 그 아래 층은 전부 다시 만들어집니다.** 그래서 '자주 바뀌는 것을 아래에' 두는 것이 핵심 원칙입니다.

REFERENCE

이 시간에 쓰는 명령

명령	하는 일	언제 쓰나
`FROM`	바탕이 될 이미지 지정	항상 첫 줄
`WORKDIR /app`	이후 명령의 기준 폴더	cd 대신 사용
`COPY <출발> <도착>`	호스트 파일을 이미지 안으로	소스를 넣을 때
`RUN`	이미지를 만드는 중에 실행	설치·빌드 명령
`EXPOSE 3000`	이 포트를 쓴다는 표시	문서용 · 실제 개방은 -p 가 함
`CMD ["node","server.js"]`	컨테이너 시작 시 실행할 명령	보통 마지막 줄
`docker build -t 이름:태그 .`	이미지 만들기 (마지막 . 은 빌드 컨텍스트)	Dockerfile이 있는 폴더에서
`-v 볼륨이름:/경로`	볼륨 연결	컨테이너보다 오래 살아야 할 데이터
`.dockerignore`	빌드에 보내지 않을 것 목록	node_modules, .git 등

FILES

실습용 파일

package.json

```
{
  "name": "myapp",
  "version": "1.0.0",
  "main": "server.js",
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

server.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("<h1>Docker 실습 v1</h1>");
});

app.listen(3000, () => console.log("listening on 3000"));
```

HANDS-ON

실습 순서

STEP 01 앱 파일 준비

이미지로 만들 아주 작은 앱을 만듭니다.

```
mkdir docker-app
cd docker-app
# 아래 '실습용 파일' 두 개를 이 폴더에 만듭니다 (package.json, server.js)
ls
```

확인 포인트

- [ ] package.json 과 server.js 두 파일이 있다

> node_modules 는 만들지 않습니다. 설치는 이미지를 만들 때 컨테이너 안에서 이뤄집니다.

STEP 02 Dockerfile 작성 (순서 최적화 전)

가장 먼저 떠오르는 순서로 써 봅니다.

```
# Dockerfile 이라는 이름의 파일 (확장자 없음)
FROM node:20-alpine
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["node", "server.js"]
```

확인 포인트

- [ ] 파일 이름이 정확히 Dockerfile 인지 확인 (Dockerfile.txt 아님)

STEP 03 첫 빌드와 실행

이미지를 만들고 띄워 봅니다.

```
docker build -t myapp:v1 .
docker images
docker run -d -p 3000:3000 --name app myapp:v1
# 브라우저에서 http://localhost:3000 접속
```

확인 포인트

- [ ] 각 단계 옆에 소요 시간이 표시된다
- [ ] 마지막에 naming to docker.io/library/myapp:v1 done 이 나온다
- [ ] 브라우저에 'Docker 실습 v1' 이 보인다

> 빌드 로그의 **[4/6] RUN npm install** 옆 시간을 기록해 두세요. 다음 단계에서 비교합니다.

STEP 04 캐시가 깨지는 지점 확인 — 오늘의 핵심 (1)

소스 한 줄만 고치고 다시 빌드해 봅니다.

```
# server.js 의 v1 을 v2 로 한 글자만 수정
docker build -t myapp:v2 .
```

확인 포인트

- [ ] COPY . . 줄부터 CACHED 표시가 사라진다
- [ ] npm install 이 처음부터 다시 실행된다
- [ ] 빌드 시간을 여기에 적어 두세요 → _____ 초

> 소스만 고쳤는데 왜 설치를 다시 할까요? **COPY . . 가 RUN npm install 위에 있기 때문입니다.** 소스가 바뀌면 그 층이 무효가 되고, 그 아래는 전부 다시 만들어집니다.

STEP 05 순서를 바꿔 다시 빌드 — 오늘의 핵심 (2)

의존성을 먼저 굳히는 순서로 고칩니다.

```
# Dockerfile 을 아래 순서로 수정
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

확인 포인트

- [ ] COPY 가 두 번으로 나뉘었고, npm install 이 그 사이에 있다

STEP 06 같은 수정으로 시간 비교

두 순서의 차이를 숫자로 확인합니다.

```
docker build -t myapp:v3 .
# server.js 의 v2 를 v3 으로 다시 한 글자 수정
docker build -t myapp:v4 .
```

확인 포인트

- [ ] v3 빌드는 처음이라 오래 걸린다 (정상)
- [ ] v4 빌드에서는 RUN npm install 줄에 CACHED 가 붙는다
- [ ] v4 빌드 시간을 적어 두세요 → _____ 초

> **기록해서 비교하세요.** 앞의 v2 빌드 시간과 지금 v4 빌드 시간의 차이가 순서 최적화의 효과입니다. 실무에서는 이 차이가 하루 수십 번 쌓입니다.

STEP 07 .dockerignore 추가

빌드에 불필요한 파일을 보내지 않습니다.

```
# .dockerignore 파일 생성
node_modules
.git
Dockerfile
*.log
```

확인 포인트

- [ ] 다시 빌드하면 transferring context 크기가 줄어든다

> 빌드 컨텍스트는 **Dockerfile이 있는 폴더 전체가 Docker로 통째로 전송되는 것**입니다. node_modules 를 빼는 것만으로도 크게 빨라집니다.

STEP 08 볼륨으로 데이터 지키기

4교시에서 사라졌던 데이터를 살려 봅니다.

```
# 볼륨 없이
docker rm -f app
docker run -d -p 3000:3000 --name novol myapp:v4
docker exec novol sh -c "mkdir -p /app/data && echo 주문1 > /app/data/order.txt"
docker exec novol cat /app/data/order.txt
docker rm -f novol && docker run -d -p 3000:3000 --name novol myapp:v4
docker exec novol cat /app/data/order.txt
# 볼륨을 붙여서
docker rm -f novol
docker run -d -p 3000:3000 -v mydata:/app/data --name withvol myapp:v4
docker exec withvol sh -c "echo 주문1 > /app/data/order.txt"
docker rm -f withvol
docker run -d -p 3000:3000 -v mydata:/app/data --name withvol2 myapp:v4
docker exec withvol2 cat /app/data/order.txt
docker volume ls
```

확인 포인트

- ☐ 볼륨 없이 만든 파일은 컨테이너를 지우면 No such file 이 된다
- ☐ 볼륨을 붙이면 새 컨테이너에서도 '주문1' 이 그대로 읽힌다
- ☐ docker volume ls 에 mydata 가 남아 있다

> 볼륨은 컨테이너 바깥에 있는 저장 공간입니다. **DB 컨테이너에 볼륨이 필수인 이유**가 바로 이것입니다.

STEP 09 정리하기

실습 자원을 정리합니다.

```
docker rm -f withvol2
docker images
docker volume ls
```

확인 포인트

- ☐ myapp 이미지가 v1~v4 까지 쌓여 있는 것을 확인했다

> 태그를 붙이지 않고 빌드를 반복하면 이름 없는 <none> 이미지가 쌓입니다. 태그는 습관으로 붙이세요.

TROUBLESHOOTING

자주 만나는 오류

증상	원인	해결
`COPY failed: file not found`	빌드 컨텍스트 밖의 파일 또는 경로 오타	Dockerfile이 있는 폴더 기준으로 경로 확인, 명령 끝의 . 확인
빌드는 되는데 브라우저 접속 안 됨	EXPOSE만 쓰고 -p 를 안 붙임	docker run -p 3000:3000 으로 실행
매번 `npm install` 을 다시 함	COPY . . 가 RUN npm install 위에 있음	의존성 파일을 먼저 COPY 하도록 순서 변경
`npm ERR! ENOENT package.json`	WORKDIR 또는 COPY 경로 문제	WORKDIR 설정 후 COPY package*.json ./ 확인
이미지 용량이 너무 큼	node_modules까지 복사 · 무거운 베이스 이미지	dockerignore 추가 + alpine 계열 사용 + 멀티스테이지(6교시)

REVIEW

복습 체크리스트 — 답할 수 있으면 통과

- ☐ 레이어 캐시가 깨지면 그 아래 줄들은 어떻게 되나요?
- ☐ 자주 바뀌는 파일은 Dockerfile의 위쪽과 아래쪽 중 어디에 두어야 하나요?
- ☐ EXPOSE 만 쓰면 브라우저에서 접속할 수 있나요?
- ☐ 컨테이너를 지워도 데이터를 남기려면 무엇이 필요한가요?
- ☐ .dockerignore 를 쓰는 이유를 두 가지 말해 보세요.

한 줄 정리

Dockerfile은 실행 순서를 적은 문서가 아니라 캐시 전략을 적은 문서입니다. 안 바뀌는 것을 위에, 자주 바뀌는 것을 아래에 두세요.

다음 시간 → 6교시 — 컨테이너 하나로는 부족할 때

WORKSHEET

막힌 STEP 번호, 화면에 나온 오류 메시지, 강사에게 물어볼 것을 적어 두세요.